TP : Seuillage d'images et prédiction du seuil par Machine Learning

1. Problématique

Le seuillage binaire transforme une image en niveaux de gris en deux classes de pixels. Pour un seuil T, les pixels d'intensité inférieure ou égale à T appartiennent à une classe, les autres à la seconde. La méthode d'Otsu calcule automatiquement un seuil à partir de l'histogramme.

Dans ce TP, vous étudierez si des modèles supervisés peuvent apprendre à prédire un seuil à partir de caractéristiques extraites d'images synthétiques.

Point méthodologique. Le seuil cible des images synthétiques sera défini à partir des intensités connues lors de leur génération. Les modèles apprendront donc une règle sur ce jeu artificiel. Cette expérience ne suffit pas à conclure qu'ils surpassent Otsu sur des images réelles.

2. Environnement

Utiliser Jupyter Notebook ou un fichier Python. Vérifier la disponibilité des bibliothèques :

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Si nécessaire : `pip install numpy opencv-python matplotlib scikit-learn`.

3. Partie A — Générer les images et construire le dataset (20 min)

Générez automatiquement au moins 100 images synthétiques en niveaux de gris. Chaque image doit comporter un fond et une forme (rectangle ou disque) dont l'intensité diffère de celle du fond. Faites varier les intensités, la taille de la forme et, si possible, ajoutez un faible bruit.

Pour chaque image, calculez et enregistrez :

- la moyenne des intensités ;
- l'écart-type des intensités ;
- la proportion de pixels d'intensité inférieure ou égale à 127 ;
- le seuil calculé par Otsu ;
- le seuil cible T^* défini à partir des intensités du fond et de l'objet connues lors de la génération.

Consignes

1. Fixez une graine aléatoire pour rendre l'expérience reproductible.
2. Affichez quelques images générées et leurs histogrammes.
3. Constituez X avec les caractéristiques et y avec les seuils cibles T^*.
4. Séparez les données en apprentissage (75 %) et test (25 %), avec `random_state=42`.

Questions A

1. Pourquoi faut-il plusieurs images pour entraîner et évaluer un modèle supervisé ?
2. Quelle est la différence entre une caractéristique (entrée X) et une cible (sortie y) ?
3. Pourquoi ce dataset synthétique ne représente-t-il pas nécessairement les difficultés des images réelles ?

4. Partie B — Calculer le seuil d'Otsu (10 min)

Pour chaque image, calculez le seuil d'Otsu avec OpenCV. Affichez l'image originale, son histogramme et le résultat du seuillage.

```
seuil_otsu, image_binaire = cv2.threshold(
    image, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

Questions B

1. Pourquoi le seuil fourni à `cv2.threshold` est-il fixé à 0 avec `THRESH_OTSU` ?
2. Le seuil d'Otsu est-il nécessairement égal au seuil cible T^* ? Justifiez.
3. Dans quelles situations un seuil global peut-il produire une segmentation imparfaite ?

5. Partie C — Entraîner les modèles supervisés (25 min)

Entraînez deux modèles de régression pour prédire T^* à partir des caractéristiques de l'image :

- **Modèle 1 :** régression linéaire ;
- **Modèle 2 :** Random Forest Regressor avec 100 arbres et `random_state=42`.

```
modele_lr = LinearRegression()
modele_lr.fit(X_train, y_train)

modele_rf = RandomForestRegressor(
    n_estimators=100, random_state=42
)
modele_rf.fit(X_train, y_train)
```

Consignes

1. Prédisez les seuils sur X_{test} pour les deux modèles.
2. Calculez la MAE, la RMSE et le coefficient R^2.
3. Présentez les résultats dans un tableau comparatif.
4. Tracez les valeurs réelles T^* en fonction des valeurs prédites pour chaque modèle.

Questions C

1. Quel modèle obtient la plus petite MAE ?
2. Pourquoi sépare-t-on les données en apprentissage et en test ?
3. Une meilleure performance sur ce dataset signifie-t-elle que le modèle sera meilleur sur toutes les images ?

6. Partie D — Comparaison et discussion (20 min)

Sur les mêmes images de test, comparez le seuil cible T^*, le seuil d'Otsu, la prédiction de la régression linéaire et celle de Random Forest. Pour chaque méthode, calculez l'erreur absolue par rapport à T^*. Affichez au moins trois exemples avec l'image originale et les résultats binaires.

Construisez un tableau de synthèse avec : méthode, seuil obtenu, erreur absolue par rapport à T^* et commentaire visuel sur le résultat.

Questions D

1. Les modèles peuvent-ils prédire un seuil hors de l'intervalle $[0, 255]$? Comment traiter une telle prédiction ?
2. Pourquoi une faible erreur sur le seuil ne garantit-elle pas toujours la meilleure segmentation ?
3. Quelles limites sont dues au caractère synthétique des images ?
4. Que faudrait-il modifier pour évaluer sérieusement ces méthodes sur des images réelles ?

7. Livrables

— Le notebook Python contenant le code et les réponses.

— Un tableau comparatif des performances (MAE, RMSE, R^2 pour les modèles et erreur absolue du seuil pour chaque méthode).

— Au moins trois visualisations commentées.

— Une conclusion courte indiquant ce que l'expérience permet — et ne permet pas — d'affirmer.

8. Barème indicatif sur 20

Critère	**Points**
Génération des images et construction du dataset	4
Calcul et visualisation du seuil d'Otsu	3
Entraînement des deux modèles	5
Évaluation et comparaison des résultats	4
Interprétation et limites	4

Conclusion attendue : distinguer le calcul statistique direct d'Otsu de l'apprentissage supervisé d'une règle à partir d'exemples, et discuter les limites d'un dataset synthétique.